数据库系统原理与实验：SQL查询语言深入学习

基于您提供的课件内容，我将全面讲解SQL查询语言的核心概念和操作，结合期末考试的考点进行分析。

一、SQL概述与特点

SQL的基本特征

SQL（结构化查询语言）具有三大核心特点：

一体化特性

- 功能一体化：集数据定义（DDL）、数据操纵（DML）和数据控制（DCL）于一体
- 操作对象单一化：所有操作都针对关系（表）进行

高度非过程化

- 用户只需指定"做什么"，无需关心"怎么做"
- 数据库管理系统自动完成操作实现过程

面向集合的操作方式

- 一次一集合：每次操作的对象和结果都是关系（记录集合）
- 与传统编程语言的"一次一记录"方式形成对比

SQL的组成部分

- **DML** ：数据操纵语言，用于查询和更新数据
- **完整性约束** ：保证数据的一致性和正确性
- **视图定义** ：创建虚拟表
- **事务控制** ：管理事务的开始和结束
- **嵌入式SQL** ：在编程语言中嵌入SQL语句
- **授权** ：指定关系和视图的访问权限

二、SQL数据定义功能

数据定义语句概览

课件中提供了完整的数据定义操作矩阵：

域类型（数据类型）

SQL支持丰富的域类型：

- 字符串类型：`char(n)`（定长）、`varchar(n)`（变长）
- 数值类型：`int`、`smallint`、`numeric(p,d)`（定点数）
- 浮点类型：`real`、`double precision`、`float(n)`

创建表（CREATE TABLE）

基本语法结构：

```
CREATE TABLE r (
    A1 D1,
    A2 D2,
    ...,
    An Dn,
    <完整性约束1>,
    ...,
```

```
    <完整性约束k>
);
```

示例（创建教师表）：

```
CREATE TABLE instructor (
    ID char(5),
    name varchar(20) NOT NULL,
    dept_name varchar(20),
    salary numeric(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department
);
```

完整性约束类型

1. **主键约束**：`PRIMARY KEY (A1, ..., An)`
2. **外键约束**：`FOREIGN KEY (Am, ..., An) REFERENCES r`
3. **非空约束**：`NOT NULL`

三、SQL查询功能详解

基本查询结构

SQL查询的基本形式：

```
SELECT A1, A2, ..., An
FROM r1, r2, ..., rm
WHERE P
```

运算次序：FROM（笛卡尔积）→ WHERE（选择）→ SELECT（投影）

SELECT子句详解

SELECT子句近似于关系代数中的投影运算，但功能更丰富：

基本功能

- 选择特定属性：`SELECT name FROM instructor`
- 消除重复：`SELECT DISTINCT dept_name FROM instructor`
- 选择所有属性：`SELECT * FROM instructor`

高级功能

- 算术表达式：`SELECT ID, name, salary/12 AS monthly_salary`
- 属性重命名：`SELECT name AS instructor_name`
- 字面量查询：`SELECT '437' AS FOO`

重要注意事项

- 默认情况下（使用 `ALL`或不指定），结果保留重复元组
- 只有使用 `DISTINCT`时才消除重复，此时等价于投影运算
- 多表查询时，同名属性必须加关系名前缀

WHERE子句详解

WHERE子句对应关系代数中的选择运算，用于筛选满足条件的元组。

条件表达式支持的操作

- 比较运算符：<, <=, >, >=, =, <>
- 逻辑运算符：AND, OR, NOT

- 范围判断：`BETWEEN 下界 AND 上界`
- 集合判断：`IN (集合)`, `NOT IN (集合)`
- 字符串匹配：`LIKE '模式'`（使用 `%`和 `_`）

特殊运算符

- 空值判断：`IS NULL`, `IS NOT NULL`
- 元组比较：`(instructor.ID, dept_name) = (teaches.ID, 'Biology')`

FROM子句与连接操作

FROM子句指定查询涉及的关系，执行笛卡尔积运算。

多表连接示例

```
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;
```

关系重命名

- 语法：`旧关系名 AS 新关系名`或`旧关系名 新关系名`
- 自连接时必须使用重命名

四、高级查询功能

集合运算

SQL支持标准的集合运算：

- `UNION`：并集（自动消除重复）
- `INTERSECT`：交集
- `EXCEPT`：差集

保留重复版本的集合运算：

- `UNION ALL`、`INTERSECT ALL`、`EXCEPT ALL`

聚集函数与分组查询

常用聚集函数

- `AVG()`：平均值
- `MIN()`/`MAX()`：最小/最大值
- `SUM()`：求和
- `COUNT()`：计数

分组查询（GROUP BY）

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

分组条件（HAVING）

HAVING子句用于对分组后的结果进行筛选：

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

WHERE vs HAVING的区别

- WHERE在分组前筛选元组，作用于单个元组
- HAVING在分组后筛选组，作用于整个分组

排序（ORDER BY）

ORDER BY子句用于对最终结果排序：

```
SELECT DISTINCT name FROM instructor ORDER BY name DESC;
```

可以按多个属性排序，每个属性可以单独指定升序（ASC）或降序（DESC）。

五、嵌套查询与高级特性

子查询类型

子查询可以出现在多个位置：

1. **WHERE子句中**：构造复杂条件
2. **FROM子句中**：创建派生关系
3. **SELECT子句中**：标量子查询

集合成员资格检查

- `IN`：检查是否属于某个集合
- `NOT IN`：检查是否不属于某个集合

示例：查询2017年秋季开课且2018年春季也开课的课程

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
AND course_id IN (
    SELECT course_id
    FROM section
    WHERE semester = 'Spring' AND year = 2018
);
```

集合比较

- `> SOME`：大于子查询结果中的某个值
- `> ALL`：大于子查询结果中的所有值

存在性测试

- `EXISTS`：子查询结果非空时返回真
- `NOT EXISTS`：子查询结果为空时返回真

相关子查询示例：

```
SELECT course_id
FROM section AS S
WHERE semester = 'Fall' AND year = 2017
AND EXISTS (
    SELECT *
    FROM section AS T
    WHERE semester = 'Spring' AND year = 2018
    AND S.course_id = T.course_id
);
```

六、数据库更新操作

删除数据（DELETE）

```
-- 删除特定记录
DELETE FROM instructor WHERE dept_name = 'Finance';

-- 使用子查询的删除
DELETE FROM instructor
WHERE salary < (SELECT AVG(salary) FROM instructor);
```

插入数据（INSERT）

```
-- 插入单条记录
INSERT INTO course
VALUES ('CS-437', 'Database Systems', 'Comp.Sci.', 4);

-- 插入查询结果
INSERT INTO instructor
SELECT ID, name, dept_name, 18000
FROM student
WHERE dept_name = 'Music' AND total_cred > 144;
```

更新数据（UPDATE）

```
-- 简单更新
UPDATE instructor SET salary = salary * 1.05;

-- 条件更新
UPDATE instructor SET salary = salary * 1.05 WHERE salary < 70000;

-- 使用CASE语句的复杂更新
UPDATE instructor
SET salary = CASE
    WHEN salary <= 100000 THEN salary * 1.05
    ELSE salary * 1.03
END;
```

七、重点考点分析与真题对应

根据期末考试真题，SQL部分的主要考点包括：

选择题重点

- 关系代数与SQL的对应关系（如×运算符对应FROM子句）
- 自然连接要求有共同属性
- 数据库规范化解决插入、删除异常和数据冗余

填空题重点

- 数据字典/元数据的概念
- 相关子查询的定义
- 授权语句（GRANT/REVOKE）
- 外连接类型（左外连接、全外连接）

综合题重点

第1题：关系代数与SQL的相互转换

- 关系代数表达式与SQL语句的等价写法
- 涉及选择、投影、连接等基本操作

第2题：函数依赖与范式

- 识别函数依赖
- 求候选码
- 判断范式等级及分解

第3题 ：查询优化

- 连接操作的I/O代价计算
- 嵌套循环连接与块嵌套循环连接

设计题重点

- E-R图设计（实体、属性、联系）
- E-R图向关系模式的转换

八、学习建议

1. **理解运算顺序** ：牢记FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY的执行顺序
2. **区分相似概念** ：

- WHERE vs HAVING
- IN vs EXISTS
- 传统集合运算 vs 保留重复的集合运算

1. **多练习复杂查询** ：特别是涉及多层嵌套、相关子查询的题目
2. **注意细节** ：空值处理、字符串匹配、数据类型转换等容易出错的地方
3. **结合真题练习** ：重点掌握综合题中出现的查询模式

这份学习指南全面覆盖了课件中的SQL知识点，并结合期末考试的考点进行了重点强调。在学习过程中，建议结合实际操作来加深理解，特别是复杂查询的编写和执行计划的分析。